Ownership for In-Place Unobservable Updates in Lean 4

Without Modifying the Compiler

Gavin Mendel-Gleason ✉
Scidonia Limited

Abstract

Copy-on-write gives Lean 4 in-place array updates indistinguishable from updating an immutable array: when the array is unshared—reference count one—the write mutates existing storage, and otherwise it copies, so only the cost changes. We turn that condition into a static guarantee for a compiler-IR fragment, by a count-one ownership discipline mechanised against model semantics. Its foundation is an accounting relation equating each count with the fields that reference it plus the owning variables that denote it, which at count one forces one owning root with no edge, or one edge with no root. Preservation is proved for a restricted flat fragment, and for extraction, call and restoration at its accounted boundaries, where the temporary pair makes the relation false. For the recognised fragment, the counted interpreter's whole run is proved heap-compatible with the model run and to agree on result, fault and trace, under an explicit counted child-call contract. The result is a checked count one relative to recognition, layout, entry and the child-call contract, and on the certified shapes the linked probe observes the emitted C deciding as predicted.

2012 ACM Subject Classification Theory of computation → Logic and verification; Software and its engineering → Software verification and validation

Keywords and phrases reference counting, ownership, compiler verification, Lean 4, separation logic

1 Introduction

Lean 4 arrays are pure functional values, and copy-on-write updates them in place without that being observable: when the array is unshared—reference count one, and not persistent—a write mutates the existing storage, and otherwise it copies, so the result is indistinguishable from an immutable array's and only the cost changes [3, 9, 10]. That cost is not invariant: Lean's `lean_is_exclusive` decides at run time, from heap state, so n updates over an n-element array can cost linear or quadratic time according to sharing alone. A static ownership discipline would make the in-place branch a compile-time guarantee; Lean 4 has no ownership or linear types, and adding them would be a language change. This paper recovers the guarantee without one, for a fragment of the compiler's IR, by an external count-one discipline whose soundness is proved in Lean 4 against model semantics.

The first obstruction is that the raw invariant does not establish ownership accounting. It equates the heap's stored count with the context's multiplicity, but binding a second owned variable leaves that equality true while two variables now denote the cell; a kernel-checked countermodel (`alias_not_accounted`) exhibits exactly that state. The repair is complete accounting: a duplicate-free support enumerating every owning variable, a domain covering every live cell, and the equation $rc(r) = \mathit{IncomingCount}(r) + \mathit{EnvOwnerCount}(r)$, whose right side counts the heap fields that reference r and the owning variables that denote it. At count one the equation admits exactly two shapes—one owning root and no incoming edge, or no owning root and exactly one incoming edge—and two owning variables at one cell are the same variable. The integer now has a meaning.

The second obstruction is the emitted code. A nested update takes the element out of the array, allocates a pair holding parent and child, projects both handles, frees the pair, calls on the child and puts the result back. At the take-out the pair's field duplicates the


© Gavin Mendel-Gleason;
licensed under Creative Commons License CC-BY 4.0


parent reference without raising the count, while the caller's handle still owns it, so the equation reads one = one plus one; it becomes true again at entry, after the release and after restoration, and at the context the abstract take-out returns no support accounts the pair-live states in between. The proof is therefore a boundary simulation: count identities for the pair's traffic, restoration's in-place branch derived from the protocol and call boundaries, and a compatibility relation that lets the counted cursor run one ahead of the model's. Closing the fragment needs a modular boundary. The child call is a contract, and its counted version must also state the call's effect on the caller's incoming edges; from that effect and the caller's accounting at the call, the accounting at the body's return is derived. The finite-path induction over model paths does not construct that edge effect. For a body that is itself the take-out/call/restoration chain it is no longer a premise: the chain is not edge-preserving step by step, but its net run is, and `counted_nested_chain_edges` proves that the entry and the restored counted heap have the same incoming-edge count at every name. Composing that effect at a second level is not claimed: the inner call's isolated accounting would drop the suspended parent's owning root while its count-one cell remains in the heap, which is inconsistent (`isolated_accounting_impossible`), so the composition statement is recorded as a regression and the net effect stands on its own.

We prove, for one level of a finite supported path under that contract, that the counted interpreter's run over the whole recognised fragment—extraction, child check, call, restoration and return—is related to the model's run of the same term by heap compatibility, with the same result, fault and write trace (`principal_counted_fragment`). A second theorem composes recognition, the finite-path model correspondence and the counted boundary accounting (`principal_supported_path`), and a closed kernel-checked instance inhabits it at the recogniser's layout. The closest predecessor, Huisinga's static uniqueness analysis for a Lean IR model [4], enforces uniqueness by a static analysis and leaves soundness and the compiler connection as future work; here the analysis is a proved relation over an existing IR, and a recognised emitted declaration's trace is interpreted and related to the model. The guarantee remains relative to entry, layout, recognition and call premises, and is stated at the observation boundary of cells, result, fault and trace: the allocator cursor is the only component that differs, and no client reading those sees it.

Three contributions.

- **Complete accounting and its preservation.** An owning-variable support, a live-cell domain and the roots-plus-edges equation yield the count-one dichotomy, reject the two-owner countermodel, and make preservation provable: `supported_run` takes an accounted state and one supplied slot fact and concludes a fault-free run of the supported fragment with exclusively flagged writes and accounting at every state reached: a move transfers an owner, a single decrement removes a root with its count, and a scalar store adds and removes no edge.
- **A counted protocol at its accounted boundaries.** The pair's traffic is neutral at release, the negative controls leave counts elevated, and restoration's mutate branch is derived rather than assumed; `counted_nested_chain` composes take-out, an admitted child call and restoration into one chain, related to the model by a cursor-tolerant compatibility relation. The technique is what survives a correct protocol under which the simple invariant is deliberately false: count identities inside the window, accounting at its boundaries.
- **A checked connection from emitted LCNF to the models.** A recogniser walks the actual declaration, pinning operation order, arities, operand identities and branch

constructors; its trace is executed and related to the model, at the designated flat write and, with the counted chain, over the whole recognised fragment at one supported level, with a closed instance discharging every premise.

The rest of the paper follows that order: the example and the two languages (Section 2), accounting (Section 3), the checker and flat preservation (Section 4), the counted protocol (Section 5), the compiler connection (Section 6), the instance and evidence (Section 7), and the comparison and limits (Sections 8 and 9).

2 The running example and the semantic setting

Lean 4 compiles source through a sequence of IRs. Its last LCNF phase inserts reference-count operations and the backend emits C; `lean_is_exclusive` tests a stored count against one and a persistence flag. The object here is single-threaded, and count one with non-persistence is the whole of the test we model. The development pins the toolchain, the sibling revision and the captured LCNF artifacts, so every claim names the code it is about. The problem is not to implement reference counting but to know statically that the count at a particular write is one; since Lean's source has no linearity, we state the property over the compiler's code and prove the connection, rather than extend the type system.

2.1 The update and its ownership traffic

The source is the array case of `Layout5.setAt` in the sibling: take the element out, check the extracted focus, recurse on it, put the result back.

```
| _, .index i q, y, s =>
    if h : i < s.size then
      match takeOutAt (assertExclusiveArray openSite s) i h (stateDefault _) with
      | ⟨(sib, focus), hbound⟩ =>
        plugAt sib i hbound (setAt q y (assertExclusive structSite focus))
    else s
```

In three lines, the update is

```
let (parent, child) = takeOut s i
let child' = setAt child 0 v
putBack parent i child'
```

The emitted form has the same shape, with the exclusivity assertions and the temporary pair made explicit: a `lean_is_exclusive` test for the array, the take-out exchange (`lean_array_fget`, `lean_array_fset` and the pair allocation), the child check on the extracted element, the recursive call, and the `lean_array_fset` put-back. Figure 1 draws the same three states throughout the paper: the entry, where one root and one slot edge account for the two cells; the take-out, where the temporary pair holds the parent in a field while the caller's handle still owns it; and the release, where the pair is gone and the two projected handles own the cells again.

2.2 Two languages, one accounted subset

The checker does not run on Lean source or on LCNF. Its programs are two small inductive types, interpreted over a model heap and environment; the compiler connection is a mapping from recognised LCNF to these values.

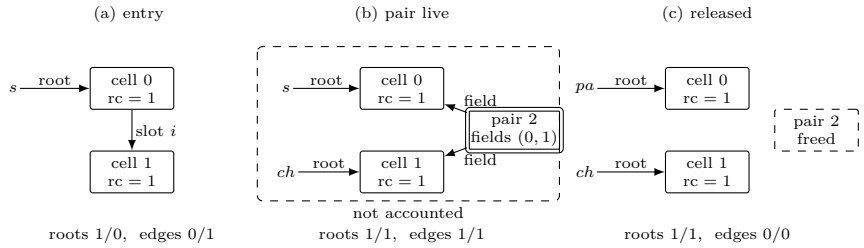


Figure 1 Ownership traffic for one nested update. Each panel is one state of the protocol of Section 5; the pair is the fresh name 2 and i is the selected slot. Counts are per cell, printed as roots over edges for parent and child. The middle panel is outside the accounting relation of Definition 2: the pair's field is an incoming edge for cell 0 while s is still an owning handle for it, so the equation reads $1 = 1 + 1$; the kernel refusal is `pairPhase_refused`. The first and third panels are accounted, at the entry and at the protocol's boundary.

```
inductive Code where
  | let (x : Var) (v : LetF) (k : Code)
  | inc (x : Var) (n : Nat) (k : Code)
  | dec (x : Var) (n : Nat) (k : Code)
  | check (x : Var) (k : Code)
  | setArr (x : Var) (i : Var) (v : Var) (k : Code)
  | ret (x : Var)

inductive NCode where
  | base (c : Code)
  | takeOut (a i fst snd : Var) (e : Ref) (k : NCode)
  | putBack (a e i result : Var) (k : NCode)
  | callAt (state path x result : Var) (body : NCode) (k : NCode)
  | ret (x : Var)
```

A `Code` is the straight-line array-write fragment; an `NCode` adds extraction, restoration and the recursive call. A `LetF` is one of `lit n`, `size a`, `isShared x` or `checkAlias x`. Variables are `Nat`, references are `Nat`, `Val` is a reference, a scalar or erased, and a `Heap` maps references to objects with a field array, a count and a persistence flag. `run`/`runN` execute these values and `check`/`checkN` are the checkers; every soundness theorem is about this semantics. They are neither Lean source nor LCNF: the recogniser walks the LCNF value, the mapping reads a program off the recognised operands, and the executor runs the recognised trace.

The two languages play different roles. `Code` and `NCode` are the checker's *general* language, with increments, multi-count decrements and arbitrary nesting, accepting programs that carry no accounting. The theorem-bearing fragment is `Supported` (Section 4.2), an inductively defined subset of `Code` with extra certificates. Checker acceptance, supported certification, state validity and the child-call specification are separate notions; accounting is proved for the certified fragment only.

3 Complete ownership accounting

The model heap, its counts and exclusivity are the semantic side of the judgement: `Inv` ties the context's multiplicity to the heap count, and `Exclusive H r` means that `r` is a live, non-persistent cell whose count is one.

▶ **Definition 1** (The raw invariant). *`Inv H env c` equates the heap's stored count with the context's multiplicity (`rc_eq`); an owning variable denotes its reference at multiplicity at least one (`owned`); a borrowed variable also denotes a live cell (`borrow`); a scalar variable denotes a scalar (`scalar`); and no live cell is persistent (`nonPersistent`).*

`Inv` relates the heap count to the context multiplicity but does not count the environment's owning variables: `Inv.owned` only requires $1 \leq c.mult\ r$. `bindOwned` preserves `Inv` without changing `mult`, so two owned variables can denote one count-one cell, and the raw invariant does not imply physical uniqueness. The kernel countermodel `alias_not_accounted` constructs exactly that state from `initCtx`, which has one owner and does satisfy `OwnersAccounted` (`initCtx_accounted`). `CountFidelity` is not a consequence of `Inv`; it is a consequence of the relation below.

▶ **Definition 2** (Complete accounting)

```
structure Accounted (support : List Var) (H :
    Heap) (env : Env) (c : Ctx) : Prop where
  inv              : Inv H env c
  support_nodup    : support.Nodup
  support_sound    : ∀ x, x ∈ support → c.us x = .owned
  support_complete : ∀ x, c.us x = .owned → x ∈ support
  support_bound    : ∀ x, x ∈ support → ∃ r, c.refOf x = some r
  domain           : ∀ r o, H.cells r = some o → r < H.next
  count            : ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c support r
```

The relation is deliberately redundant, because each field of it is refutable: a duplicate in the support counts a root twice (`dupSupport_witness`), an omitted owner loses one (`omittedOwner_witness`), and an unbound root has no denotation to count (`unboundRoot_witness`). The support is a duplicate-free enumeration of every owning variable, each entry is bound to a reference, every live cell lies inside the range the edge count enumerates, and the count equation holds over two independently computed quantities.

```
def IncomingCount (H : Heap) (r : Ref) : Nat :=
  ((List.range H.next).map fun q =>
    match H.cells q with
    | some o => (o.fields.toList.filter (fun v => v = .ref r)).length
    | none => 0).sum

def EnvOwnerCount (c : Ctx) (vars : List Var) (r : Ref) : Nat :=
  (vars.filter (fun x => c.us x = .owned ∧ c.refOf x = some r)).length

def CountFidelity (H : Heap) (c : Ctx) (vars : List Var) : Prop :=
  ∀ r, H.rc r = IncomingCount H r + EnvOwnerCount c vars r
```

▶ **Lemma 3** (Count-one dichotomy). *If `Accounted support H env c` and $H.rc\ r = 1$, then either $EnvOwnerCount\ c\ support\ r = 1$ and $IncomingCount\ H\ r = 0$, or $EnvOwnerCount\ c\ support\ r = 0$ and $IncomingCount\ H\ r = 1$.*

Proof. Read the count equation through `Inv.rc_eq` (`mult_eq`) and split the two natural numbers whose sum is one. `count_one_dichotomy` is the statement. ◀

▶ **Corollary 4** (Uniqueness at a known root). *If `Accounted support H env c`, $H.rc\ r = 1$, and an owning variable $x \in support$ has $c.refOf\ x =$ some r, then x is the only owning variable at r and r has no incoming edge; `at_most_one_owner` states that any two such variables coincide.*

This is the count-one consequence that gives the integer a meaning, and `initCtx` establishes the relation at the initializer (`initCtx_accounted`). It is a statement about owning variables and heap edges, not about physical pointers: no representation theorem connects a model reference to a runtime object.

Rule	Obligation	Effect
`let y (checkAlias x)`	`x` owned, multiplicity one; `y` not owned	`y` becomes the checked alias; `x` dead
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`dec x 1`	`x` owned; $1 <$ multiplicity	multiplicity -1; `x` dead
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; no open frame at `a`	push the frame; bind parent and element
`callAt state path x result b`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree b`; body accepted from the isolated context and returns `state`	call the body; bind the result

Table 1 Selected checker rules. Each premise is an ownership obligation; the successor context is computed, not inferred.

4 The checker and flat preservation

4.1 The judgement, once

A context records, for each variable, whether it is owned, borrowed, scalar or dead, which reference it denotes, and the multiplicity of that reference. The checker is a total function from a context and a program to a successor context or failure; a frame stack records open extractions.

```
inductive U where
  | owned | borrow | scalar | dead

structure Ctx where
  us    : Var → U
  refOf : Var → Option Ref
  mult  : Ref → Nat
```

A frame records the array, index and element of an open extraction; `check : Ctx → Code → Option Ctx` and `checkNF : Ctx → List Frame → NCode → Option Ctx` are total.

The judgement.

The typing judgement $c \vdash s \Rightarrow c'$ is *defined* to hold exactly when `check c s = some c'`: the context is the type of the program point, and there is no separate derivability relation to relate to it. The object language is untyped; there is no type former for a program, and the checker does not infer one by unification. The judgement is syntax-directed—one rule per constructor, the context an input, the successor computed by `setUs`, `move`, `bindOwned`, `inc` and `dec`—so there are no metavariables and the only computation is the context update. Table 1 gives the rules the rest of the paper uses; the remaining constructors (literal, size, sharing, check, put-back and return) have the same shape: a usage-and-multiplicity premise, and a successor computed by those updates, with `ret` refusing a frame it does not close.

Two consequences matter later. `dec` is affine, not the last drop: its premise is $n <$ the object's multiplicity, so it consumes a duplicate handle and leaves the object live. And linearity at the write is the side condition *multOf* $x = 1$, not the usage tag alone.

▶ **Example 5** (A unique write is accepted, an aliased one refused).

```
def execSet (x y i v : Var) : Code :=
```

```
    .let y (.checkAlias x) (.setArr y i v (.ret y))

def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails; the same refusal appears at the context level for a heap whose count is two. This is the static counterpart of the runtime copy: the checker rejects the program before it can copy.

4.2 The supported fragment

`Supported c s` is indexed at the pre-state `c`; each constructor states the checker's own rule at `c` and its continuation at the checker's post-state, and carries the certificates the checker cannot see: a fresh alias target and a fresh scalar-`.let` target. The fragment has eight constructors; `.inc` is absent and every `.dec` is the single-count form `.dec x 1`, whose premise `1 < c.mult r` is exactly the checker's. From `Supported c s` the checker accepts the program (`supported_check`), which is what lets a preservation theorem follow a fault-free run.

The store's slot fact is deliberately not a constructor premise: no context records which value a cell's selected slot holds, so it is the `StoreSlots` premise of the preservation theorem instead. `StoreSlots H env s` is a recursive predicate over the supported program, supplied from the heap and environment alone (it may not assume the invariant): at the store the run reaches, the array cell's selected slot holds a scalar, so the in-place write replaces a value rather than an edge. `storeSlots_init` discharges it for the initializer's measured write, and nothing general.

4.3 Preservation

▶ **Theorem 6** (Supported execution preserves accounting). *Let `Supported c s`, `Accounted support H env c` and `StoreSlots H env s`. Then there are c', H', env', v and ws with `check c s = some c'` and `run s env H = .done H' env' v ws`, such that every write in ws is flagged exclusive, `Accounted (resultSupport s support) H' env' c'`, and `AccountedAlong H env c support s`.*

The slot fact concerns the store's old selected slot as well as the scalar written, so reference-valued array updates lie outside this preservation theorem. `AccountedAlong` is a conclusion, never a hypothesis: at each instruction the relation holds at the interpreter's own post-state, and at each array store the target is exclusive (`supported_store_exclusive` states the store's own pre- and post-state).

Proof sketch.

Induct on the derivation of `Supported c s`, generalising the support, the heap and the environment; the measure is the syntax of the supported program and the induction hypothesis is applied to the continuation at the state the step produces. Each case is a step of `run` followed by the induction hypothesis, and the content is in three cases. *Fresh binding* (`bindAlias`): the alias target takes over the source's owning obligation, and the support variable is renamed by `replaceVar`; `Accounted.move` shows that `EnvOwnerCount` is unchanged, because the

enumeration follows the handle rather than the name. Freshness is what makes this a transfer: were the target already an owner, the support would gain a duplicate and `support_nodup` would fail. The scalar binders (`bindLit`, `bindSize`, `bindShared`) use `Accounted.setScalar`, which adds no owner and no edge. *Decrement* (`decOne`): the support loses `x`, the heap loses one count, and `IncomingCount` is unchanged, so both sides of the equation at r fall by one (`Accounted.decOne`); the multi-count decrement is excluded precisely because it would release counts that no obligation carried (`decTwo_rejected`). *Edge-neutral scalar store* (`setArr`): the slot fact gives that the selected slot holds a scalar, and the run writes a scalar, so the set of incoming edges is literally unchanged; the equation is preserved by `Accounted.setArrInPlace` at the checker's unchanged context, which is where the write's exclusivity and its trace event are produced. The remaining cases commute with the induction hypothesis.

5 Counted protocol and boundary simulation

5.1 The pair and its boundary

The take-out builds a pair of the parent and the extracted child; the projections increment both components; freeing the unique pair releases both edges. At the boundary the counts return to their entry values only because the pair is freed.

▶ **Theorem 7** (Pair accounting). *Let `WellFormedEntry H p c i`: both components live, non-persistent and at count one, p's selected slot holding c, and the fresh name empty. Let B = protocol H p c i return the fresh name B_1 and the heap B_2. Then*

$$B_2.rc\ p = 1, \quad B_2.rc\ c = 1, \quad B_2.cells\ B_1 = \text{none},$$

both components are `Heap.Exclusive` in B_2, and their counts are H's.

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧ B.2.cells p ≠ none ∧
      B.2.cells c ≠ none ∧ B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

The no-free control, `no_free_components_not_exclusive`, leaves both components at count two, neither exclusive, and the pair live: the boundary is produced by the free, not by the projections.

The table's component roots are the take-out's obligations in the context the abstract take-out returns; the pair's own cell is a heap object with no owning variable there.

5.2 The counted call and restoration

`counted_takeOut` consumes the array's cell and selected slot plus the context facts from `checkNF`; it derives the boundary accounting, `HeapCompat`, the pair's freshness and the cursor advance. The child call is where the modular boundary sits. `CountedChildCall` adds one field to the model child-call contract:

```
structure CountedChildCall (Hin Hout : Heap) (env : Env) (ctx : Ctx)
    (state path x : Var) (body : Nested.NCode) (p c : Ref)
    (support : List Var) : Prop where
```

Stage	`EnvOwnerCount`	`IncomingCount`	parent rc	child rc	disposition / why
entry	1 / 0	0 / 1	1	1	root handle; child in the parent's slot
take-out	1 / 1	1 / 1	1	1	handles moved; pair fields are the edges
projections	1 / 1	1 / 1	2	2	`incPairComponents_eq`: edge plus handle
free unique pair	1 / 1	0 / 0	1	1	`pair_traffic_neutral`: edges released
child call	1 / 1	0 / 0	1	1	`CountedChildCall`: parent suspended
put-back	1 / 0	0 / 1	1	1	`putBack_exclusive_of_wellFormed`: mutate

Table 2 The counted protocol at an accounted caller entry. Each root/edge pair is printed ordered parent/child. The take-out row is the state of Figure 1(b), where the stored count and the equation disagree: the row's counts are per-stage effects, not boundary facts. The boundary rows are the first, the free and the put-back; `Accounted` is asserted only there.

```
call  : PutBack.ChildCall Hin Hout env ctx state path x body p c
edges : EdgePreserved Hin Hout
```

`call` is the boundary the put-back consumes. Unfolding it, the body must be admitted, the checker must accept it from the isolated context and return the call's own reference, the isolated caller invariant, the annotations and the extraction obligations must hold, the parent must lie outside the child's mutable footprint, and the body must complete on the caller's heap returning that reference with its count and persistence unchanged and its effect confined to the child's footprint. Those are the ordinary call contract, and the finite-path induction derives them at every level (`childContracts_of_path`). `edges` states the call's effect on the caller's heap: every reference keeps its incoming-edge count, the cursor does not go backwards, and no cell appears where the entry had none. It is the semantic fact a counted body is responsible for; the boundary alone does not imply it, because a scalar store over a '.ref' slot of the child's own cell can erase an edge into a caller-owned cell while every field of the call boundary and the confinement fact hold (checked as a negative control beside the harmless rewrite). From `edges` and the caller's accounting at the call, `accounted_of` derives the caller's complete accounting at the body's return; the induction does not construct the edge effect, because the ordinary contract never mentions incoming edges. The closed instance supplies it from the leaf's scalar rewrite and computes the same accounting directly (`calledAccounted`), and the body it runs is a model computation; the emitted recursive callee remains an implementation obligation. From the call's output state, `counted_restore` derives the mutate branch and the accounted post-state, and `putBack_exclusive_of_wellFormed` concludes `Heap.Exclusive` for both the parent and the restored child.

▶ **Theorem 8** (Counted chain). *From an accounted caller entry, take-out, an admitted `CountedChildCall` and restoration compose into one chain: `counted_nested_chain` concludes the final counted state, its accounting, the restoration's `.mutate` disposition, and `HeapCompat` to `runN`'s state; the restored slot's own equation is `counted_nested_chain_restoration`. The chain is not edge-preserving step by step, but its net run is: `counted_nested_chain_edges` concludes `EdgePreserved` between the entry and the restored counted heap, which is the entry-to-output effect that `counted_nested_chain`'s final-state*

`HeapCompat` does not give. Composing the effect at a second level is not claimed: the inner call's isolated accounting would drop the suspended parent's owning root while retaining its count-one cell, which is inconsistent (`isolated_accounting_impossible`); the composition statement is recorded as a regression, and the net effect stands.

5.3 Compatibility and the boundary argument

▶ **Definition 9** (Heap compatibility). *`HeapCompat Hc Ha` holds when Hc and Ha hold the same cell at every name and $Ha.next \leq Hc.next$. At the protocol instance the slack is exactly the pair allocation the counted side performs and the model's take-out does not; the relation itself permits any slack, and `HeapCompat.eq_of_cursor_eq` shows that with equal cursors the heaps are equal, so it is equality of cells up to the counted side's allocation, not a weakened equivalence.*

In place throughout is the modelled disposition: the array operation neither copies nor allocates under the contracts these statements take, while the protocol's take-out allocates the temporary pair and the counted cursor records it.

The recorded trace is the list the interpreters report: the child body's stores and the protocol's own scalar events. The take-out exchange and the restoration's array store are justified separately — the first by the boundary accounting, the second by the derived disposition and its target's exclusivity — rather than by appearing in that list, so trace equality is evidence about recorded events and the store coverage comes from those statements.

The pair-live window is a boundary abstraction, and it is worth naming the interpretation it refuses. During the window we use the context the abstract take-out returns, whose two owning variables are the take-out's obligations for the parent and the extracted child; the pair itself is the fresh heap cell and is owned by no variable of that context. At the take-out state of Figure 1(b), at exactly that context, no support satisfies `Accounted` (`pairPhase_refused`), because the count equation reads one = one plus one. A finer intermediate relation could root the pair and treat its projections as borrowed; the boundary relation instead keeps one invariant at the entry, the release and the restoration. Three facts make the window safe. The projections' effect is exactly the pair's component increment (`incPairComponents_eq`). The release is boundary-neutral (`pair_traffic_neutral`): the parent's and child's counts are their entry counts, the child's cell is the entry's, and the temporary pair is gone. And the suspended parent remains exclusive across the child call because the child's contract is confined to the child's mutable footprint rather than because the parent's exclusivity is assumed: `callAt_frame_confined` takes `AnnotationsIn`, checker acceptance, the return variable, the isolated `Inv` and `TakeOutsOK`, and the parent/child separation is a `ChildCall.separated` fact used in the path derivation. Finally the put-back's mutate branch follows from the protocol boundary and the child-call boundary, not from an assumed final state.

Proof sketch.

The protocol argument is a decomposition, not an induction: the five heap-touching operations are discharged in order, each by a count identity on the heap it produces, and the composition is `counted_nested_chain`. Transfer into the pair moves the handles and leaves the pair's fields as the edges; projection adds one root per component; release removes the pair's two edges and nothing else; the call is suspended at the boundary; restoration reads the child's output state and writes the slot. Because each identity is stated at the state it produces, the chain never needs `Accounted` inside the window—only at its ends—and `HeapCompat` carries the one-cell cursor slack that the allocation leaves behind.

6 Recognised-fragment correspondence

6.1 Recognition, and the flat guarantee

A source-level discipline is only useful if it survives lowering. The compiler rewrites programs, inlines helpers and chooses the emitted shape, so the recogniser walks the actual LCNF value, pinning operation order, arities, operand identities and branch constructors. `projectPrefix` accepts the flat write, `projectNested` the `Path.index` arm and `projectPair` the two pair arms; token-preserving near misses are rejected (eleven at the top level, eleven on the nested path, fourteen per pair arm), so the correspondence is about the code we name. No universal rejection theorem is claimed; the claim is scoped to the accepted grammar and the mutation corpus. `project_sound` gives executed/model equality for the abstract prefix, and `execAST_recognized` is decoding adequacy: it relates the trace to the model, it is not a semantics.

▶ **Theorem 10** (Flat principal guarantee). *Let `projectPrefix code = some p`, `Layout assign p`, the entry invariant `EntryInv H env (Runner.initCtx root) root client`, the machine/model agreement `Agrees assign p s env H root i v`, the cell and index facts* $H.cells\ root = \text{some}\ o$ *and* $i < |o.fields|$*, and a runtime correspondence `Corr H R` with* $R.cells\ root = \text{some}\ o'$*. Then the executed prefix's outcome and the model's run both equal `inPlaceOutcome root i v o H`, and for some* H' *the runtime's `setAt` returns `.inPlace H' root` with `Corr` preserved, the target root and the slot holding the written scalar.*

```
theorem principal_write
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v) (hcell : ...) (hin : i < o.fields.
    size)
    (hc : Corr H R) (ho' : R.cells root = some o') :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
      PrefixOutcome.ofModel (Ownership.run (Prefix.toModel assign p) env H) =
        inPlaceOutcome root i v o H ∧
      ∃ H', Runtime.setAt R root i (.scalar v) = some (.inPlace H' root) ∧ ...
```

The proof is the conjunction of `recognized_write_in_place` and `Runtime.setAt_inPlace_of_entryInv`, and it takes no accounting premise. `Accounted` is consumed at the initializer by `principal_accounted`, which yields `OwnersAccounted`, `CountFidelity` and the count-one dichotomy. `Prefix.toModel_supported` puts the certified flat model in the counted fragment, so `supported_run` applies once `StoreSlots` is supplied—at `execSet` by `storeSlots_init`, not for a general *p.toModel*. The two results are adjacent, not one theorem.

6.2 The finite path and the counted fragment

The model nests take-out, call and put-back per level. For a finite level path the ordinary child contract is derived at every level: `childContracts_of_path` takes the level list and a supplied path contract whose one function-level ingredient is the leaf computation (agreement, checker acceptance, exit variable, count-freedom, annotation), the rest being the level's slot, environment, context, shape and heap facts, and yields the ordinary `ChildContract` at each level, whose body is the remaining levels' `Path.build`; `index_path_simulates_model` concludes that the executed nested prefix's `NestedOutcome` is the model run of `Path.build path value body (l :: rest)`, completing with an all-exclusive trace under the recogniser's verdict, layout, level match, agreement and path contract. The finite model does

not interpret successive path-node projections by the emitted recursive callee: a level after the first takes its index from one of the two scalars the call exposes, while the recogniser re-projects the path each invocation holds. The recognised outer fragment is related to a supplied model child computation, and the emitted callee's frame and path projections remain outside the correspondence.

The flat and nested guarantees take different premises: the flat one takes the entry invariant and the runtime-model correspondence and concludes no accounting, while the accounting results take `Accounted`. They are adjacent theorems, not one preservation result for every recognised prefix.

▷ Claim 11 (in words). Let the caller's state be accounted, the recogniser accept the emitted declaration, and the child call satisfy its contract with its edge effect. Then the counted run of the recognised fragment returns the caller's array handle with no fault, writes only to exclusive targets, and leaves a heap agreeing with the model's run cell for cell; only the allocator cursor differs.

The connection has four layers, and they are separate claims. Structural recognition pins operation order, arities, operands and branch constructors. Decoding consistency makes the trace decode to the expected prefix, so the recogniser's verdict is spent by the execution theorem rather than assumed. Interpreter correspondence relates the counted run of that trace to the model's run of the built path: same cells, result, fault and trace. Supplied meanings name the modelling boundary — the take-out store, the child body and the recursive callee enter as contracts, and the C primitives are observed, not proved — so recognition is not execution adequacy of the LCNF or C.

▶ **Theorem 12** (Counted fragment correspondence). *Let the recogniser accept the declared shape (`projectNested` yielding p), let the layout, the machine-to-model agreement `NestedAgrees` and the level match hold at one level of a finite path, let the client's path contract and the caller's complete accounting `Accounted support H mEnv c` hold, and let the counted child call be supplied at the protocol's boundary with its `EdgePreserved` effect on the caller's heap (the caller's accounting at the body's return is derived from that effect). Then, writing H_f for the counted final heap:*

1. *`execCountedAST` runs the whole recognised fragment p.fragment from the entry machine to a completed state with heap H_f, result the caller's array handle, restored binder holding that handle, and write trace the entry trace followed by the child's;*
2. *the model run of `Path.build path value body (l :: rest)` completes with that handle and an all-exclusive trace;*
3. *the counted and model outcomes are `NestedOutcome.Compat`: they agree by `HeapCompat` and literally on the result, the fault and the write trace;*
4. *the accounting holds at the protocol's boundary over the extended support, and at H_f over the support the call's result binding and the restoration's renamings determine;*
5. *at the boundary the fresh cursor cell is empty, the cursor advances by one, both components are exclusive, and the boundary heap is heap-compatible with the abstract take-out heap; at the call's output heap the put-back selects `mutate`, writes the child into the parent's selected slot, and changes no other cell.*

Client observations are what the relation leaves free. Write `Observed` for an outcome's cells, result, fault and write trace; the allocator cursor is not part of it, because addresses are not observable in the source. The theorem's `NestedOutcome.Compat` conjunct fixes every component of that observation, so no client reading cells, result, fault and trace can separate

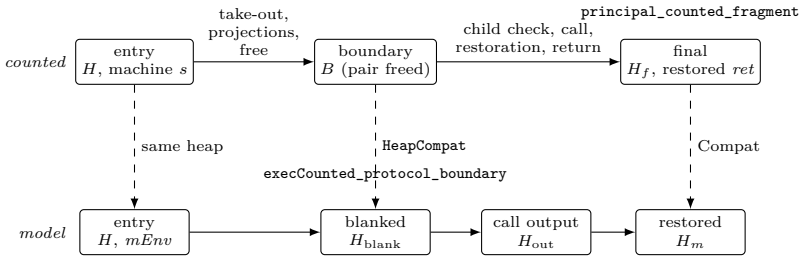


Figure 2 Simulation boundaries. The counted and model rows are connected only where a theorem relates them: at entry, after the pair's release (`execCounted_protocol_boundary`) and after restoration (`principal_counted_fragment`). Inside the pair-live window the accounting is not an invariant of the counted execution, so no vertical edge is drawn there; the suffix is run from the boundary state, and its restoration is derived at the call-output heap H_{out}.

the counted run from the model's (`NestedOutcome.Compat.observed_iff`). The in-place update is therefore indistinguishable from the functional one at this boundary, and the cursor is the only component that differs.

The full type, all conjuncts and the premise providers are in `OwnershipLcnf/Principal.lean`.

Proof sketch.

The measure is the decomposition of the fragment, not a recursion on syntax. Recognition fixes operand identities and operation order, so the counted interpreter's fragment is a fixed sequence; `fragment_decomp` splits it at the release-terminated prefix `countedPrefix`—a syntactic split, not a numeric cutoff—giving `countedSuffix` as the rest. On the prefix, `execCounted_protocol_boundary` reports the protocol boundary heap, agrees with `execNested` on the non-heap fields including the write trace, and relates the two heaps by `HeapCompat`. On the suffix, `execCounted_full_fragment` runs the child assertion, the call at the boundary state, the restoration at the call's output heap and the return. The call is consumed where the interpreter reaches it, so the supplied contract is used at the actual call-entry state rather than at a reconstruction of it, and the restoration's mutate branch is derived at the actual output state. The two halves compose to `NestedOutcome.Compat`, and because the entry point is `execCountedAST`, the recogniser's verdict is spent by the theorem rather than assumed by its statement.

7 The instance, the case study and the artifact

7.1 A worked instance

The instance is a family. For an outer slot i, a leaf slot j and a value w, the entry holds two live, non-persistent cells at count one: the parent array at reference 0 carries the child handle in slot i and scalars elsewhere, the child array at reference 1 is all scalars, and reference 2 is the fresh name. The child body is one designated write that stores w in the child's slot j and returns the child handle. The family is proved end to end for every $i < n$ and $j < n$: the entry is accounted, the boundary reports its cells and its accounting, the counted child-call contract is discharged, and the whole chain's conclusion holds, with the length-two instance as its corollary. The bound is the only length fact the proofs use: the slot facts are read off the array library's bounds lemma, and the count equation follows from the take-out's edge-fill lemmas on the all-scalar base heap.

Take $i = 0$, $j = 1$ and $w = 7$. Before the take-out the parent's slot 0 holds `.ref 1` and its slot 1 a scalar; after it, slot 0 holds the placeholder `.scalar 0`, the temporary pair has been allocated at name 2 and released, and the cursor stands at 3. The leaf then stores `.scalar 7` in the child's slot 1, and the restoration refills the parent's slot 0 with `.ref 1`. Three facts are separate and none implies the others: the restored array is functionally the model's own `set`; the counts are back to one at both components with the pair gone; and the restoration took the `.mutate` branch because the parent is exclusive at the call's output state, not because the counts happen to be unchanged. The final heap and the abstract run's heap are related by `HeapCompat`, not equality: the counted run's cursor is one higher, the residue of the temporary pair.

Both composed theorems are instantiated, closed, at the captured declaration's data: every premise is supplied at the recogniser's layout, and the instances' axiom footprint is the trusted one. The counted instance discharges the child-call contract's accounting field from the caller's own accounting at the body's return, so the contract is met at the instance rather than assumed by it.

Five controls probe distinct mechanisms, and they are not interchangeable. A retained parent of count two makes the restoration copy rather than mutate, the relation exposing the extra allocation through the cursor; an omitted pair release leaves both component counts elevated with no exclusivity; the pair-live state has no accounting at the take-out context. In those three the relation refuses. A duplicated, omitted or unbound owner entry shows another face — a duplicate double-counts a root, an omitted owner still satisfies the equation, an unbound entry counts nothing — so each support field is load-bearing. A persistent parent is refused by the checker instead: the check aborts and no write is reported, which is a different mechanism from a refusal by the relation, and a copy branch in the model is not an allocation claim about the executable. Each witness is named in the artifact documentation.

7.2 Evidence and artifact

The proof checks are kernel builds of `Ownership`, `OwnershipTest` and `OwnershipLcnf`, an axiom audit over the headline theorems, and the artifact gate that binds the pinned LCNF, C and header. They are separate categories from the structural recogniser controls (the near-miss counts and the control fixtures described above) and from the linked runtime observations. A compiled probe compares the model's predicates with `lean_is_exclusive` and address identity on a corpus including a unique array, a live alias, the alias dropped back to one, a persistent literal and a closure-held array; its differential mode compares recorded expectations rather than executing the `Ownership` model, and its closed-pure-array case is an unchecked observation recorded in `docs/notes/OWNERSHIP_RUNTIME_EVIDENCE.md`. On the shapes the discipline certifies, the probe observes the alignment directly: the unique and nested cases keep the buffer and the written index at the same address and the write lands, while a live second reference changes the address and leaves the outside reference's values intact; with the check armed the harness reports instead of writing. The theorems are the proof and the probe is observation. The declarations behind every claim, with their roles and the paper-to-code entry points, are listed in the supplementary document `docs/notes/ownership-theorem-map.md`, which is artifact documentation and not part of the argument.

The evidence answers different questions: the parameterised family covers one model level but is not an application of the recognition theorem; the closed instance covers the whole counted-fragment theorem with a supplied model child; the structural controls reject the specified malformed traces only; the linked probes observe exclusivity, address and results on

the recorded corpus; and the artifact gate checks the pinned sibling and its captures.

The discipline makes a cost prediction, and it is observable. `Accounted` with count one forces the exclusive, in-place disposition (`setArrInPlace`, `inPlace_of_exclusive`), so a certified write should take the constant-cost branch and a shared one the copying branch. On a hand-written union-find over `Array Nat` — outside the fragment, checked by no rule here — the unshared workload costs a flat 66–75 ns per iteration as the array grows from 2 000 to 16 000 elements, while a variant that keeps a snapshot live at the write costs 559–5 288 ns per iteration over the same growth: the shared workload grows with the array size while the unshared one stays flat over the measured range. These are whole-iteration figures, not per-store latencies. The union-find results are identical, so sharing changes the cost and not the value; the emitted C calls the runtime's copy-on-write setter, whose branch is the reference-count test itself. This observes the predicted branch; it is not a cost theorem, because the refinement from the model heap to the runtime is the contract above. In the other direction, a checker-accepted model program is emitted as Lean and built, and its printed array agrees with the model interpreter; measurements and caveats are in `bench/README.md`.

Applying the development to a changed source update is four steps: pin the sibling and recapture, which fails if the recogniser's prefix changes; run `projectNested`; supply the client facts — entry accounting, layout, agreement and level data, the path contract, and the child call's edge effect; then check the certificate with the axiom audit and the claim census. The third step is the manual one; the rest is automatic. The same condition can be checked on a surface of Lean terms before compilation: an experimental borrow/move analysis accepts union-find's threading and refuses a retained alias or a capture, but it tracks binders rather than owners and is not a sound usage checker; `Accounted` remains the semantic half.

8 Related work

Huisinga's 2023 thesis [4] is the closest predecessor and the comparison is exact. It analyses a model of Lean's IR, enforces uniqueness through a static analysis, and studies that analysis's checker; uniqueness there is a property of the analysis, not of the runtime count; it has no compiler link; and its soundness argument and compiler integration are stated as future work. The difference is not that our proofs are machine checked—theirs could be too—but what is connected: here a count-one checker is proved against a model heap with an explicit roots-and-edges accounting, that result is composed with a transform of the recognised emitted prefix and with a second model of the write's runtime state (`Runtime.RHeap`, distinct from the linked Lean runtime) at the flat write, and the extraction protocol is isolated at proved accounted boundaries. This is not a claim to the first static uniqueness analysis for Lean.

Perceus [9] and Counting Immutable Beans [10] transform programs by inserting counts and reuse tokens and prove the transformation preserves semantics; neither asks when a count of one is justified at a write in existing code, which is what `Accounted` specifies and what this paper proves for a fragment. Linear Haskell [2] makes linearity a source-level type property and Clean's uniqueness typing [1] is uniqueness rather than linearity; neither checks an existing compiler IR or connects to emitted code. Lean's `Array.markLinear` [6] is dynamic: it can copy an array to establish uniqueness and, with `LEAN_ABORT_ON_NONLINEAR`, abort on a later nonlinear use. Translation validation [8] checks one compiler run, and verified compilation [7, 5] proves a compiler correct for its source language; both differ in kind from a checker over a fragment of an existing backend's input. The reusable insight is that count-one reasoning is a boundary discipline: a count of one is not one owner, so the

accounting needs a support and a live-cell domain; a correct temporary-alias protocol is not an invariant, so the pair-live window is covered by count identities with accounting at its boundaries.

The five differ on every axis: Huisinga certifies an analysis over a Lean IR model, without soundness proof or compiler link; Perceus and Counting Immutable Beans certify transformations with a resource calculus; Linear Haskell and Clean certify source-level type properties. This paper certifies a recognised fragment of emitted code, with roots-and-edges accounting, a proved correspondence to the model, and an explicit child-call contract the caller discharges, changing nothing in the compiler.

9 Scope, conclusion and disclosure

What is checked, proved, supplied and assumed is kept separate. Checked syntax is established by `#guard` tests and the recogniser. Model facts are theorems in `Ownership` and `OwnershipLcnf` with no axioms beyond Lean's. Client and call facts — the entry invariant, agreement, layout and path contract, the leaf computation, and `CountedChildCall`'s `edges` effect, from which the post-call accounting is derived — are hypotheses of the two principal theorems, discharged at the closed instance. Implementation assumptions are fixed by the runtime contract and observed by the linked gate.

Conclusion.

A count-one discipline can be recovered for an existing compiler IR without changing the language, provided the accounting is complete rather than local and the emitted protocol is treated as a boundary phenomenon rather than an invariant. That is what this paper establishes: a proved relation between owning roots and incoming edges, preservation for a restricted flat fragment, a counted extraction, call and restoration protocol proved at its accounted boundaries, and — for one level of a finite supported path — a checked correspondence to the model's run under a child-call contract that the closed instance discharges. The result buys a checked count one at the model heap and the recognised trace; on certified shapes the linked probe observes the emitted C deciding as predicted. Next come the C semantics, replay and address identity, free and cascade, and the emitted callee.

Acknowledgements

AI assistance.

The development and this manuscript were produced with substantial AI assistance. Tools and model versions used across the development were not recorded. The model identifier `deepseek/deepseek-flash` is recorded for this manuscript revision in `docs/notes/artifacts/final-revision-check.txt`. Categories of assistance: prose, Lean proofs, code, analysis and literature search. The division of labour between the author and these tools is not recorded. Outputs were checked by the pinned kernel build and the axiom audit, and a declaration census checks each displayed name against the development; no line-by-line human review of the proof scripts is recorded. The author exercised editorial control over the submitted text and evidence; kernel checking is not human validation, and the definitions, theorem statements, interpretation of evidence, novelty claims and citations are the author's responsibility.

References

1. Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 6(6):579–612, 1996.
2. Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In *Principles of Programming Languages (POPL)*, 2018.
3. Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE-28*, volume 12699 of *LNCS*. Springer, 2021.
4. Marc Huisinga. Static uniqueness analysis for the lean 4 theorem prover. Master's thesis, Karlsruhe Institute of Technology, 2023. URL: `https://pp.ipd.kit.edu/uploads/publikationen/huisinga23masterarbeit.pdf`.
5. Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In *Principles of Programming Languages (POPL)*, 2014.
6. Lean FRO. Array.marklinear, 2025. Lean API documentation. URL: `https://lean-lang.org/doc/api/Init/Data/Array/Basic.html#Array.markLinear`.
7. Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 2009.
8. George C. Necula. Translation validation for an optimizing compiler. In *Programming Language Design and Implementation (PLDI)*, 2000.
9. Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In *Programming Language Design and Implementation (PLDI)*, 2021.
10. Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019.